Training MACE and TopoBench on OpenQDC datasets

A practical guide to the scripts in `scripts/openqdc/` (repo: TopoBench, branch ucsb)

1. What you get

Every OpenQDC potential-energy dataset can be trained with two models through the same pipeline: **TopoBench** graph/simplicial/cell neural networks, and **MACE** (an equivariant interatomic potential). Both read the data through one shared loader, so they see identical molecules, in the same units, with the same energy column.

Goal	Script	What it does
Write dataset configs	generate_configs.py	Writes configs/dataset/graph/openqdc_<name>.yaml, one per dataset. The repo holds 7: qm9, multixcqm9, qm7, qm7b, alchemy, vqm24, tmqm.
Train TopoBench	train_topobench.py	Creates the dataset config if needed, downloads data, runs python -m topobench.
Train MACE	train_mace.py	Exports train/valid/test .xyz files, then runs the official mace_run_train.
Only export .xyz	export_xyz.py	The export step of train_mace.py on its own.
Download + preprocess only	prepare_data.py	Checks the dataset and does what is missing (download, graphs or .xyz); both training scripts call it first.
Run on a GPU or CPU	sub_openqdc.sh	Submission script: one command to train MACE or TopoBench on a chosen GPU or on the CPU (section 6).

Shared code: `topobench/data/utils/openqdc_utils.py` (loading, unit handling), `topobench/data/datasets/openqdc_dataset.py` (PyG graphs for TopoBench) and `topobench/data/loaders/graph/openqdc_datasets_loader.py` (the TopoBench loader).

2. Before you start

- Run everything from the repository root: `/gpustorage2/lukab/TopoBench`.
- Use the project environment: `source .venv/bin/activate` (or call `.venv/bin/python` directly). It already contains topobench, mace-torch and openqdc.
- Pick a GPU that is free: `export CUDA_VISIBLE_DEVICES=1` (check with nvidia-smi; some GPUs on this machine have been nearly full).
- Data cache: OpenQDC downloads go to `$OPENQDC_CACHE_DIR` if set, otherwise `datasets/openqdc/` in the repo (gitignored). To use another disk: `export OPENQDC_CACHE_DIR=/some/absolute/path` *before* starting Python.

```
cd /gpustorage2/lukab/TopoBench
source .venv/bin/activate
export CUDA_VISIBLE_DEVICES=1
```

3. Units and what is being predicted

- **Units:** every dataset is loaded with OpenQDC's own conversion, `energy_unit='ev'` and `distance_unit='ang'` (constants in openqdc_utils.py). The values OpenQDC returns are used as they are; the raw files in the cache are never modified.
- **Known OpenQDC unit problem (not corrected):** OpenQDC labels the QM9 and QM7b energies as hartree, but the numbers are kcal/mol (checked on methane against its published U0 of -40.4789 Ha). Converted with OpenQDC's own eV factor they come out about 627 times too large (QM9 molecule 0: -7,223,240 eV instead of about -11,400 eV). Other datasets may have similar label errors; check a molecule you know before trusting a new dataset.

- **One energy per molecule.** Datasets store several energies per molecule, one column per level of theory (QM9: 9 columns, QM7b: 76, Alchemy: 1). **You choose the column in the dataset yaml**: set `loader.parameters.energy_method` in `configs/dataset/graph/openqdc_<name>.yaml` to the index of the column (the header of the file lists index and name for every column). TopoBench and MACE both read that value. You can override it per run with `--energy-method <index or name>` (or `dataset.loader.parameters.energy_method=<index>` for Hydra). For QM9, column 0 is the standard U0.
- **Target:** `formation_energies` (default), `energies` (total) or `per_atom_formation_energies`. For datasets without isolated-atom reference energies (QM9, QM7b) formation and total energies are identical.
- **Per-element baseline:** molecular energies are large numbers dominated by the atoms. MACE subtracts fitted per-element energies itself; TopoBench does the same (`baseline: per_element`), so its error is comparable to MACE's total-energy error. The baseline only changes the training target `y`; the original energy is kept as `data.energy`. Set `baseline=none` to train on the raw energy.
- **Forces:** training is *energy-only*; forces are never loaded.

Choosing the energy column: example

Open the dataset yaml and set `energy_method` to the index of the column you want (the list at the top of the file shows index and name):

```
# configs/dataset/graph/openqdc_qm9.yaml
loader:
  parameters:
    # Energy column to train on: its index in the list at the top of this file.
    energy_method: 3        # 3 = HF:sto-3g
```

Both pipelines then use that column without any flag; a flag overrides the yaml for one run:

```
# no flag: both use the column from the yaml (3)
python scripts/openqdc/train_topobench.py --dataset QM9 --model graph/gcn
python scripts/openqdc/train_mace.py --dataset QM9

# a flag overrides the yaml for this run, by name or by index
python scripts/openqdc/train_mace.py --dataset QM9 --energy-method "B3LYP:def2-svp"
python scripts/openqdc/train_topobench.py --dataset QM9 --energy-method 0
```

The chosen column is printed at the start of a run (`Energy column 3: 'HF:sto-3g'`). Indices start at 0 (QM9: 0 to 8, QM7b: 0 to 75, MultixcQM9: 0 to 228; the one-column datasets only have 0). An index and a name are interchangeable: `--energy-method 3` and `--energy-method "HF:sto-3g"` select the same QM9 column. An index outside the valid range stops the run with an error that lists the first column names. To rewrite the yaml with a given column: `python scripts/openqdc/generate_configs.py --datasets QM9 --energy-method 3`.

4. Train TopoBench

One command. Before training starts, the script checks that the dataset is downloaded and converted to graphs for exactly these settings, and does whatever is missing (details below). Then it trains.

```
python scripts/openqdc/train_topobench.py \
    --dataset QM9 --model graph/gcn --energy-method 0 \
    --max-samples 20000 --epochs 50
```

Download and preprocessing: checked automatically

Every training run starts with a check, in this order, and does only what is missing:

- **Dataset yaml** exists (otherwise it is written; the data is downloaded to detect the elements).
- **OpenQDC data** is downloaded and complete. The files are checked against each other (sizes, number of atoms, molecules and energies). A download that was interrupted is detected: the partial files are removed (message `[repair]`) and fetched again.
- **TopoBench graphs** exist for this dataset, energy column, sample size, baseline and edge settings; otherwise they are built.

Progress is printed as `[prepare] ...` lines. The same step can be run alone, for example once before starting many jobs:

```
python scripts/openqdc/prepare_data.py topobench --dataset QM9 \\
    --energy-method 3 --max-samples 20000
python scripts/openqdc/prepare_data.py topobench --dataset QM9 --loader-param baseline=none
python scripts/openqdc/prepare_data.py mace --dataset QM9 --max-samples 20000
```

`train_mace.py` does the same for MACE: download, then export the .xyz files, then train. Jobs started at the same moment wait for each other (`[wait] ...`) and reuse the result. Model-specific preprocessing of the simplicial and cell lifts still happens during training; TopoBench protects it with its own lock.

Other models and extra settings

```
# simplicial and cell models (graphs are lifted automatically)
python scripts/openqdc/train_topobench.py --dataset QM9 \
    --model simplicial/scn --max-samples 20000
python scripts/openqdc/train_topobench.py --dataset Alchemy \
    --model cell/cwn --max-samples 20000

# anything after "--" is passed to Hydra unchanged
python scripts/openqdc/train_topobench.py --dataset QM9 --model graph/gcn -- \
    optimizer.parameters.lr=0.001 dataset.loader.parameters.edge_method=radius
```

Option	Default	Meaning
--dataset	required	OpenQDC dataset name, case-insensitive (QM9, Alchemy, VQM24, ...).
--model	graph/gcn	Any TopoBench model config: graph/gcn, graph/gin, simplicial/scn, cell/cwn, ...
--energy-method	from yaml	Energy column, index or name. Default: energy_method in the dataset yaml.
--target	from yaml	formation_energies | energies | per_atom_formation_energies.
--max-samples	all	Random subset of molecules. Strongly recommended first for datasets with more than ~500k molecules.
--subsample-seed	0	Seed of that subset.
--epochs / --batch-size / --seed	from config	Passed as trainer.max_epochs, dataloader batch size, seed.
--logger	csv	csv, wandb, tensorboard, ... (wandb needs a login).
--cache-dir	see section 2	Where OpenQDC data lives.
--regen-config	off	Re-detect elements/units and rewrite the yaml.

Useful dataset-config values (override with `dataset.loader.parameters.<name>=...`)

Parameter	Default	Meaning
edge_method	bonds	bonds: covalent bonds from distances; radius: all pairs within cutoff.
bond_scale / cutoff	1.3 / 5.0	Bond if d < bond_scale x (r_i + r_j) covalent radii; cutoff in Å for radius.
baseline	per_element	none to train on the raw energy (the default is none for datasets whose energies are not total energies).
atom_types	detected	Elements of the one-hot node features. Node features = one-hot + formal charge.

Where things are saved

- Converted graphs: `datasets/graph/openqdc/<Name>/<settings>/` (includes baseline.json with the fitted per-element energies).
- Train/val/test splits: `datasets/data_splits/` (random 80/10/10, seed 0).

- Run outputs and metrics: `logs/train/runs/<timestamp>/`. Metrics are MAE/MSE in **eV** on the baseline-subtracted energy.

5. Train MACE

MACE trains through its official command-line tool on extended-XYZ files. The script exports the files first (skipped if they already exist for the same settings), then runs `mace_run_train` with energy-only settings.

```
python scripts/openqdc/train_mace.py \
    --dataset QM9 --energy-method 0 \
    --max-samples 20000 --epochs 50
```

Extra mace_run_train flags go after `--`:

```
python scripts/openqdc/train_mace.py --dataset Alchemy --max-samples 20000 -- \
    --num_channels=128 --max_L=1
```

Option	Default	Meaning
--dataset	required	OpenQDC dataset name.
--energy-method	from yaml	Energy column, index or name. Default: energy_method in the dataset yaml (else 0).
--target	formation_energies	formation_energies or energies (written to REF_energy in eV).
--max-samples / --subsample-seed	all / 0	Same molecule selection as TopoBench.
--valid-fraction / --test-fraction / --split-seed	0.1 / 0.1 / 0	Random split done during export.
--epochs / --batch-size / --lr	100 / 32 / 0.01	Training settings.
--patience	30	Early stopping, in epochs.
--r-max / --max-ell / --correlation / --num-interactions / --hidden-irreps	5.0 / 3 / 3 / 2 / 64x0e + 64x1o	Model size and cutoff.
--dtype / --device / --seed	float64 / cuda / 0	Numeric precision, device, seed.
--xyz-root / --work-dir	datasets/openqdc_xyz / logs/mace_openqdc	Where files and runs are saved.
--allow-non-total	off	Needed to export datasets whose energies are not total energies (for example QM8).

Where things are saved

- XYZ files: `datasets/openqdc_xyz/<settings>/split_v0.1_t0.1_s0/{train,valid,test}.xyz`, plus export_args.json recording the exact settings and unit corrections. Each frame holds atoms, positions in Å, `REF_energy` in eV and no forces.
- Models, checkpoints, logs and results: `logs/mace_openqdc/<run name>/`. MACE prints an error table (MAE in meV) for train, validation and test at the end.
- To only export: `python scripts/openqdc/export_xyz.py --dataset QM9 --max-samples 20000` (add `--overwrite` to redo).

Energy-only quirk: MACE's loss always includes a force term. The script predicts forces but gives them zero weight, and the XYZ files contain no forces, so training is correct. Epochs take about 3x longer than a pure-energy loop, and the log shows `MAE_F=None`; this is expected.

6. Submission script: GPU or CPU

`scripts/openqdc/sub_openqdc.sh` runs either training on a GPU you choose or on the CPU, from the repo's uv environment (`.venv`). It is modelled on the sub_*.sh scripts in the home directory: the device is a command-line argument and the thread counts are set for you.

```
scripts/openqdc/sub_openqdc.sh <mace|topobench> <gpu number|cpu> \
    [arguments of the training script]
```

```
# MACE on GPU 1
scripts/openqdc/sub_openqdc.sh mace 1 --dataset QM9 --max-samples 20000 --epochs 50

# TopoBench on GPU 1 (any model config)
scripts/openqdc/sub_openqdc.sh topobench 1 --dataset Alchemy \
    --model graph/gcn --max-samples 20000

# either one on the CPU
scripts/openqdc/sub_openqdc.sh topobench cpu --dataset QM9 --model cell/cwn \
    --max-samples 2000 --epochs 5
scripts/openqdc/sub_openqdc.sh mace cpu --dataset QM9 --max-samples 2000 --epochs 5

# in the background, with a log file
nohup scripts/openqdc/sub_openqdc.sh mace 1 --dataset QM9 > run.log 2>&1 &
```

Choosing the energy column in the submission script:

```
# choose the energy column: environment variable (index or name) ...
ENERGY_METHOD=3 scripts/openqdc/sub_openqdc.sh topobench 1 --dataset QM9 \
    --max-samples 20000
# ... or the flag (the flag wins over the variable)
scripts/openqdc/sub_openqdc.sh mace 1 --dataset QM9 --energy-method "B3LYP:def2-svp"
```

	GPU (number, e.g. 1)	**CPU (cpu)**
Visible devices	CUDA_VISIBLE_DEVICES is set to that number; the script prints the GPU name and memory use first.	All GPUs are hidden (CUDA_VISIBLE_DEVICES is empty).
MACE	--device cuda	--device cpu
TopoBench	trainer=gpu	trainer=cpu
Threads (OMP / MKL)	1 by default	8 by default

- **Arguments:** everything after the device is handed to `train_mace.py` or `train_topobench.py` (see their `--help`). Arguments after a lone `--` go to `mace_run_train` (MACE) or to Hydra (TopoBench), for example `-- --restart_latest`.
- **Environment variables:** `ENERGY_METHOD=<index or name>` chooses the energy column; `THREADS=<n>` changes the thread count; `OPENQDC_CACHE_DIR` moves the OpenQDC data.
- **Errors:** with no arguments, an unknown model name or a bad device the script prints a usage message and stops with a non-zero exit code; it also stops if `.venv` is missing.
- **Energy column:** by default the one set in the dataset yaml (`energy_method`). Override it for a run with `ENERGY_METHOD=<index or name>` in front of the command or with `--energy-method` after the device; if both are given the flag wins. An index (counting from 0) and a name select the same column, for example `ENERGY_METHOD=3` and `ENERGY_METHOD="HF:sto-3g"` for QM9. Names containing spaces or colons need quotes. The chosen column is printed when the run starts.
- **Tested:** all four combinations (MACE and TopoBench, each on GPU 1 and on the CPU) ran to the end on a 500-molecule Alchemy subset. Those runs only check the wiring.

Running several jobs at once

Jobs for different models, GPUs and energy columns can run at the same time, also on the same dataset. Everything they could write at the same moment is protected:

Shared resource	Protection
OpenQDC download / loading	One process at a time per dataset (lock file in the OpenQDC cache folder); the others wait and then find the files.
Interrupted download	Files are checked before every load. Truncated or inconsistent files of a killed download are removed and downloaded again; locks are released automatically when a job dies.
TopoBench graph conversion	Lock: the first job converts, the others wait and reuse the result.
Train/val/test splits	Written to a temporary folder under a lock and renamed when complete, so nobody reads half-written splits.
MACE .xyz export	Lock; the export counts as complete only when export_args.json exists.
Dataset yaml	Lock, and the file is replaced atomically.
MACE run folder	The default run name ends with an 8-character hash of the model/training settings (lr, hidden irreps, ...), so runs that differ only in those get their own folder. A second run with identical settings refuses to start: give it its own --name.

- **Tested:** four TopoBench jobs (graph/gcn, graph/gin, graph/gat, simplicial/scn) started together on the same dataset and energy column with fresh data: all finished and only one of them converted the data. Two MACE jobs with different learning rates started together: one export, two separate run names, both finished. Two MACE jobs with identical settings: the second one stopped with a clear message.
- **Not covered:** GPU memory. Each job needs its own GPU number or enough free memory; the script prints a warning when the chosen GPU has less than 4000 MiB free (change the limit with `MIN_FREE_MB`), but still starts the job. Only the random split used by these configs was made safe; the k-fold and stratified split functions are unchanged.

7. Choosing a dataset

The seven datasets with a config in the repo. Only QM9 and Alchemy have been run through both pipelines; MultixcQM9 was run through TopoBench; the others only had their config generated:

Dataset	Molecules	Energy columns	Notes
QM9	133,885	9	Column 0 = U0. OpenQDC labels the energies hartree but they are kcal/mol, so eV values are ~627x too large. H C N O F.
MultixcQM9	133,631	229	229 functional/basis-set columns (names in the yaml header). H C N O F. Unit labels not checked.
QM7	-	80	Config exists as a placeholder but the dataset is unusable: the remote OpenQDC copy contains only zeros.
QM7b	7,211	76	Same label problem as QM9. H C N O S Cl. Choose the column deliberately.
Alchemy	202,579	1	Labels look right (total energies match the free-atom sum). H C N O F S Cl. Tested with TopoBench and MACE.
VQM24	1,104,982	1	Up to 17 atoms; large, so start with --max-samples. 10 elements. Unit labels not checked.
TMQM	86,665	1	Transition-metal complexes, up to 569 atoms, 44 elements (45 node features). Unit labels not checked.
Any other OpenQDC dataset	-	-	No config in the repo: train_topobench.py --dataset X downloads it and writes the config. Some are hundreds of GB.

Large datasets: converting millions of molecules in Python is slow and memory-hungry. Always start with `--max-samples` (for example 20000), and check the size before letting the script download a big dataset.

8. Comparing MACE and TopoBench fairly

- Use the **same dataset, energy column, target and --max-samples / --subsample-seed** in both. Then the molecules and energies are identical.
- Compare in the same unit: TopoBench reports eV, MACE reports meV (1 eV = 1000 meV). Both errors are on total energy after removing the per-element baseline.
- **The train/val/test splits still differ** (MACE: its own random split during export; TopoBench: its split files). Until they are unified, treat the comparison as approximate.

9. Quick checks and troubleshooting

Problem	What to do
"Elements [...] missing from atom_types"	The config was written without data. Run train_topobench.py with --regen-config (it downloads the dataset and rewrites the yaml).
Download starts again / cache seems empty	Cached data was removed (this happened once on this machine). Check `ls datasets/openqdc ~/.cache/openqdc`; the next run re-downloads.
Everything is downloaded twice	A run without --cache-dir / OPENQDC_CACHE_DIR used the default; keep one location (see section 2).
MACE refuses a dataset	Its energies are not total energies (for example QM8). Pick another dataset, or pass --allow-non-total if you know what you are doing.
Errors hundreds of times too large (QM9, QM7b)	OpenQDC energy-unit label problem (see section 3); the pipelines use OpenQDC's values unchanged. Use a dataset with correct labels, e.g. Alchemy.
Out of memory on the GPU	Choose a free GPU with CUDA_VISIBLE_DEVICES; lower --batch-size.
"Another run is already using ..." (MACE)	A run with identical settings is active or left its folder locked. Wait for it, or give this run its own --name.
"[repair] ... incomplete files"	A previous download was interrupted. The partial files were removed and the dataset is downloaded again; nothing to do.
"[wait] ... another job is busy"	Another job is downloading or preprocessing the same data. This job continues by itself when it is done.
wandb login prompt	Use the default --logger csv, or run wandb login.

Regenerating all configs

```
# the 7 default datasets (placeholders are written for any that are not downloaded)
python scripts/openqdc/generate_configs.py

# write the chosen energy column into a yaml (default: keep the one already there)
python scripts/openqdc/generate_configs.py --datasets QM9 --energy-method 3

# download one dataset and detect its elements
python scripts/openqdc/generate_configs.py --datasets VQM24 --download
```

`--all` writes a config for every OpenQDC dataset; `--download` needs an explicit `--datasets` list, because some datasets are very large.